

Project Report - Andrea Mussari

MAT:637943

1. Overview & how to run

Both subprojects are independent dune projects (`lib/` , `bin/main.ml` , `test/` , `sources/`), sharing the same Makefile targets. Lexers/parsers are hand-written recursive-descent to keep full control of error messages and precedence.

```
# Native (OCaml $ge$5.4, dune $ge$3.21; LLVM/clang optional, for `make compile`)
cd MiniFun && make run FILE=main.fun           # run a program
cd MiniFun && make test                         # test suite
cd MiniImp && make run FILE=factorial.imp      # interpret + emit
generated/output.ll
cd MiniImp && make test
cd MiniImp && make compile FILE=factorial.imp # mem2reg + clang ->
generated/compiled

# Docker (no local OCaml/LLVM needed)
docker build -t ct_project .
docker run -it --rm ct_project
```

The `Dockerfile` is based on `ocaml/opam:ubuntu-24.04-ocaml-5.4` , adds `llvm / clang / make` , and pre-builds both projects.

2. Fragment 1 MiniImp front-end & interpreter

Lexer: single-pass character scanner, one-character look-ahead only for `:=` . Tokens are prepended then reversed ($O(1)$ per token).

AST: three mutually recursive types `expr` / `bexpr` / `cmd` mirroring the grammar exactly.

Parser: one function per precedence level (`parse_cmd(;) → parse_bexpr(and) → parse_bnot(not,<) → parse_add(+ -) → parse_mul(*) → parse_unary(-) → parse_atom`), each using a `while` loop for left-associativity instead of left recursion. Unary minus is desugared

to `Sub(Int 0, e)` in the parser, so no extra AST node is needed. `if` always requires an explicit `else`, so dangling-else is a non-issue by grammar design.

Runtime: a single mutable `(string, value) Hashtbl.t`, `Minilmp` has no nested scopes so a flat mutable table is simpler/faster than a persistent one. Input/output variables start `Undefined`; reading/assigning `Undefined` raises at runtime. Evaluation is a direct case-by-case transcription of the big-step rules.

3. Fragment 2 MiniFun front-end & interpreter

Same design as `Minilmp` (single-pass lexer, one-function-per-precedence-level parser: `and` `→` `<` `→` `+` `-` `→` `*` `→` `not/fun/if/let/letfun` `→` `app` `→` `base`). Function application is left-associative and parsed by a loop that keeps consuming atoms. Types have a separate mini-grammar with `->` right-associative.

Environment: implemented as a hash table, but `extend_env` **copies** the table before adding a binding, rather than mutating it necessary because closures capture the environment at creation time, and a shared mutable table would let a later binding leak into an earlier closure. `letfun f x = e1 in e2` builds a `VRecClosure` that re-binds itself before binding the parameter, giving self-reference with no mutable cells. The `typo option` field on `Func / LetFun` is what lets the same AST be reused, unchanged, by both type systems (Fragments 3 and 4).

4. Fragment 3 MiniFun type system with annotations

Q Write in the report the rule(s) you devised for handling the annotations.

Both `Func` and `LetFun` were made to require a mandatory annotation, used directly as the premise type instead of being inferred:

$$\frac{\Gamma, x:\tau_1 \vdash e:\tau_2}{\Gamma \vdash \text{fun } x:\tau_1 \Rightarrow e:\tau_1 \rightarrow \tau_2} \text{(T-Fun)}$$

$$\frac{\Gamma, f:\tau_1 \rightarrow \tau_2, x:\tau_1 \vdash e_1:\tau_2 \quad \Gamma, f:\tau_1 \rightarrow \tau_2 \vdash e_2:\tau}{\Gamma \vdash \text{letfun } f x:\tau_1 \rightarrow \tau_2 = e_1 \text{ in } e_2:\tau} \text{(T-LetFun)}$$

Reading τ_1 off the syntax (instead of guessing it) turns *checking* into a purely syntax-directed algorithm with no search. The `letfun` annotation must already be a function type; `f` and `x` are both bound before checking e_1 so recursive calls type-check, and e_1 's type must match τ_2 exactly.

Syntax / data types: annotation kept as `typo option` (parser reads an optional `: <type>` via one-token look-ahead on `COLON`, so i can use the same AST). `type typo = Int | Bool | Fun of typo*typo ; env` is a plain assoc list small, short-lived, no backtracking, so `List.assoc` is simplest and shadowing is free (fresh binding just consed on front).

Typechecking function: `typecheck : env -> expr -> typo`, one case per rule, raising `TypeError` otherwise. A single helper `expect_type expected actual ctx` centralises the compare-and-fail pattern and reports **both the syntactic context** (e.g. "if branches must match") **and** the expected/actual types. Missing or bad formed annotations are rejected as type errors, not parse errors, since the grammar allows omitting them.

5. Fragment 4 Algorithm W (Hindley--Milner)

Ignores the annotations entirely and infers types from scratch, reusing the same AST.

```
type mono = TVar of string | TInt | TBool | TFun of mono * mono
type poly = Poly of string list * mono          (* forall a1..an. mono *)
type subst = (string * mono) list
type env   = (string * poly) list              (* vars bound to
polytypes *)
```

Monotypes/polytypes are kept as distinct types so OCaml itself rejects using a still-quantified type where a monotype is expected. `inst` replaces quantified vars with fresh ones (global counter) giving each *use* of a polymorphic identifier an independent copy. `gener env t` quantifies over the free variables of `t` **not** free in `env` (the standard soundness side-condition for let-polymorphism). `apply_*` push a substitution through types/polytypes (careful not to touch the polytype's own bound vars) and environments; `compose_subst s2 s1` = "apply s1 then s2". `unify` is Robinson's algorithm with an occurs check (rejects e.g. `'a = 'a -> int`). `infer : env -> expr -> subst * mono` follows the textbook W rules: fresh type variables for lambda parameters and for a `letfun` 's (initially unknown) domain/codomain; `let` infers-then-generalises (let-polymorphism), while lambda/ `letfun` parameters are bound monomorphically, as HM requires. The public `typecheck : env -> expr -> typo` intentionally has the same name/shape as `TypeChecker.typecheck`, so the two type systems are interchangeable.

6. Fragment 5 Control-flow graphs

```

type blockCode = Stmt of Ast.cmd | Condition of Ast.bexpr
type nextNode  = EOF | NextBlock of nodeId | CondSelect of nodeId *
nodeId
type node = { id; code; mutable next }
type cfg  = { nodes : (nodeId, node) Hashtbl.t; i; f }

```

`build_cfg` allocates all nodes it needs first, then wires `next` pointers by mutation needed to close loop back-edges cleanly.

Q What are the blocks (maximal/minimal/other)? Minimal: one statement or one condition per node, never a whole sequence. Chosen because Fragments 6 and 7 need per-statement precision (a maximal block would push liveness/reaching-defs tracking *inside* the block), and one CFG node maps 1:1 to one LLVM basic block in Fragment 8.

Q How do you generate the CFG for sequences? `build_cfg` returns `(entry_id, exit_id)` for any command; for `Seq(c1, c2)` the two sub-CFGs are built independently and `c1`'s exit is wired with `NextBlock` to `c2`'s entry, returning `(entry c1, exit c2)`. Because every command has exactly one entry/exit, this composition rule is uniform regardless of what `c1 / c2` are.

Q Other details. `If` allocates a condition node (`CondSelect` to both branches) plus a synthetic `skip` join node both branches wire to, keeping the "one exit" invariant. `While` allocates a condition node (`CondSelect` into the body or out to a synthetic exit) and closes the back-edge from the body's exit to the condition. `CmdParen` is transparent (adds no node).

7. Fragment 6 Annotated CFGs & data-flow analysis

Worklist-based liveness (backward) and reaching definitions (forward) over the Fragment-5 CFG.

Q How did you manage annotations? Instead of adding mutable fields to `node`, each analysis returns a **separate map** `node-id → result (rd_state NodeMap.t, or {liveIn; liveOut})` per node) this *is* the "annotated CFG". Benefits: several analyses can annotate the same unmodified CFG independently, and the Fragment-7 optimizer (which *does* mutate node code) can never leave a stale annotation on a node it just recomputes the map it needs before each pass.

Q Edge cases handled. Non-assignment nodes have empty `def`; `Condition` nodes' uses come from the boolean expression. Self-referential assignments (`x := x+1`) work automatically since `use / def` come from the same node and $liveIn = use \cup (liveOut \setminus def)$. Confluence points (branch joins, loop headers) are handled uniformly by folding over all successors/predecessors no special-casing. Back-edges don't break termination because `VarSet / DefSet` only grow monotonically over a finite lattice; a node is re-enqueued only if its state changed, and duplicate worklist entries are filtered with `List.mem`.

Q Representation for reaching definitions? A definition = the **CFG node id** that performs the assignment (`DefSet = Set.Make(Int)`), not a `(var, value)` pair the node already records which variable it assigns, so the id alone loses no information; `kill` for a definition of `v` is computed on demand as "every other node id assigning `v`".

Q Other detail. "Defined variables" (steps 2--3) is folded into the *undefined*-variable check of Fragment 7, built directly on liveness: a variable live at the CFG entry that isn't the input variable is exactly one that may be read before being assigned this avoids a third, separate must-analysis for the same practical purpose.

8. Fragment 7 Static checks & optimizations

Q Analysis for undefined variables / handling? Reuses liveness: `liveIn` at the entry node minus the input variable = possibly-undefined variables. This is deliberately a *may*-analysis (flags on any offending path, erring toward over-reporting). Handling is non-fatal one warning per variable is printed and the pipeline continues, like GCC's `-Wmaybe-uninitialized`.

Q Compiler infrastructure? No separate IR: passes mutate the same CFG in place (`Hashtbl.replace` via a small `update_node` helper that also logs the change and flips a `changed` flag). Each pass recomputes the relevant Fragment-6 analysis from the *current* CFG before rewriting, so it stays correct after earlier passes' edits.

Q Controlling passes? `optimize_pipeline` runs propagation → folding → dead-store elimination, each returning "did I change anything"; if any did, the whole pipeline repeats, otherwise a fixpoint has been reached. Every iteration and rewrite is logged.

Q Optimization strategy, and why? That fixed order was chosen because each pass unlocks the next: propagation exposes literal-only expressions for folding, folding can make a store's value or a branch static, which creates new dead-store opportunities. Iterating to a *global* fixpoint (not just once) matters because dead-store elimination changes what reaching-defs/liveness see next round, potentially re-enabling propagation/folding.

Q Other detail. Constant propagation only fires when **all** reaching definitions of a variable agree on the same literal (soundness propagating a value true on only some paths would be wrong). Dead-store elimination explicitly protects the output variable even if it looks locally dead.

9. Fragment 8 LLVM IR generation

Memory-based translation: every variable gets an `alloca i64`, every read/write is an explicit `load` / `store`. This avoids SSA construction (no ϕ -nodes, no dominance frontiers) at the cost of non-SSA output.

Q Fresh names? A mutable `counters` record (`reg`, `label`) threaded through codegen; `fresh_reg` stamps out globally-unique `%tmp.<n>` registers. Block labels reuse the CFG node id directly (`bb_<id>`), since it's already unique no separate label counter is actually needed in practice.

Q Impact of block size? Minimal CFG blocks (`\S6`) mean many small LLVM basic blocks joined by plain unconditional branches. Upside: `compile_node` is trivial and purely local, and the LLVM CFG is by construction identical to the MiniImp CFG. The resulting redundancy is deliberately left for LLVM's own `opt` to clean up.

Q SSA technique? None, by design: `make compile` runs `opt -passes=mem2reg -S ...`, which promotes eligible allocas to registers and inserts ϕ -nodes automatically reusing LLVM's own, well-tested SSA construction rather than reimplementing dominance frontiers by hand.

Q Other detail. The module exposes one function `@func(i64 %input_val)` matching the provided `wrapper.c` (`extern int64_t func(int64_t)`), which reads `stdin`, calls `in`, and prints the result giving `make compile` $\rightarrow$ object file (`llc`) $\rightarrow$ link (`clang`) $\rightarrow$ native executable. Every `Cfg.EOF` node emits `ret i64` after loading the output variable, derived purely from CFG exit-edge structure (no AST-level special-casing needed).

10. Testing

Each subproject has its own `test/test.ml` (`make test`) with a small custom pass/fail harness (no external test library). MiniFun's suite covers lexer/parser edge cases, the annotation-based checker's rule enforcement, Algorithm W's inference (incl. let-polymorphism), and evaluation. MiniImp's suite covers parsing (including rejecting invalid identifiers like ones with underscores), CFG node counts, interpreter results on `factorial.imp` / `fibonacci.imp`, and the data-flow/optimization passes.

11. Conclusion

Both languages are implemented end-to-end without parser generators or an existing SSA library: MiniFun gets an interpreter plus two interchangeable type systems (annotation-based and full Hindley--Milner), and Minilmp gets a full CFG $\rightarrow$ data-flow $\rightarrow$ fixpoint-optimization $\rightarrow$ memory-based LLVM pipeline that hands SSA promotion off to LLVM's own `mem2reg`. Throughout, the guiding choice was to keep each stage as close as possible to a direct transcription of the course's rules/equations, and to lean on existing, well-tested tooling (LLVM's optimizer) rather than reimplementing it where the fragment sheet allowed.